

LegalMate.Pk



By:

Muhammad Haroon Shahid

45669

Sameer Abid

47145

Sudais

46274

Supervised by:

Mr. Hassaan Siddique & Dr. Mansoor Alam

Faculty of Computing

Riphaah International University, Islamabad

Fall 2025

A Dissertation Submitted To

Faculty of Computing,

Riphah International University, Islamabad

As a Partial Fulfillment of the Requirement for the Award of the

Degree of

Bachelors of Science in Computer Science

Faculty of Computing
Riphah International University, Islamabad

Date: [date of final presentation]

Final Approval

This is to certify that we have read the report submitted by ***Muhammad Haroon Shahid (45669), Sameer Abid (47145), Sudais (46274)***, for the partial fulfillment of the requirements for the degree of the Bachelors of Science in Computer Science (BSCS). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelors of Science in Computer Science (BSCS).

Committee:

1

Mr. Hassaan Siddique
(Supervisor)

2

Dr. Musharraf
(Head of Department/chairman)

Declaration

We hereby declare that this document “**LegalMate.Pk**” neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanied report entirely on the basis of our personal efforts, under the proficient guidance of our teachers, especially our supervisors **Mr. Hassaan Siddique and Dr. Mansoor Alam**. If any part of the system is proved to be copied out from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

Muhammad Haroon Shahid
45669

Sameer Abid
47145

Sudais
46274

Dedication

With sincere gratefulness to Allah Almighty for granting us the strength, patience, and willpower to complete this final year project. We would like to express our deepest appreciation to our respected supervisors for their valuable guidance, continuous support, and constructive feedback throughout the development of Legal Mate. Their mentorship played a vital role in shaping this project.

We are also thankful to our team members, whose commitment, collaboration, and hard work were essential in bringing this project to end. Their promise and partnership helped transform ideas into a real-world and meaningful solution.

Acknowledgement

First of all, we are obliged to Allah Almighty the Merciful, the Beneficent and the source of all Knowledge, for granting us the courage and knowledge to complete this Project.

We would like to thank our supervisors very much for always being there with their expert support, guidance, and positive criticism, besides, the supervisor's encouragement and mastery were crucial factors in our understanding and the enhancement of our work's quality. We also owe a debt of gratitude to our teachers who were instrumental in laying the strong academic base upon which we built our final year project and for their continuous support. Friends and classmates are thanked for their collaboration, conversations, and moral support throughout the project, particularly during the idea development stage. Last but not least, we want to say that our families were always there for us, cheering up, being patient, and praying, thus giving us the strength, we needed to overcome this work successfully.

Muhammad Haroon Shahid
45669

Sameer Abid
47145

Sudais
46274

Abstract

This project presents **Legal Mate**, an AI-based legal assistance system designed to help common citizens understand basic legal concepts related to **property law**. Legal information is often complex and difficult for non-legal users to interpret, which limits access to timely and affordable legal guidance. Legal Mate aims to address this problem by providing simplified legal explanations through an intelligent chatbot integrated into a mobile application. The system utilizes modern artificial intelligence and natural language processing techniques to process user queries submitted in text, voice, or document form. Users can also generate basic legal documents and optionally connect with registered lawyers for further guidance. The proposed system focuses on improving accessibility to legal information while maintaining ethical boundaries by not replacing professional legal advice.

Table of Contents

Chapter 1: Introduction	1
1.1 Goals and Objectives	1
1.2 Scope of the Project	2
Chapter 2: Literature Review	3
2.1 Introduction.....	3
2.2 Background and Problem Elaboration	3
2.3 Detailed Literature Review	4
2.3.1 Definitions.....	4
2.3.2 Related Research Work 1.....	5
2.3.3 Related Research Work 2.....	6
2.3.4 Related Research Work 3.....	7
2.3.5 Related Research Work 4.....	8
2.3.6 Related Research Work 5.....	9
2.4 Literature Review Summary Table	10
2.5 Market Survey	11
2.6 Research Gap	12
2.7 Problem Statement	13
Chapter 3: Requirements and Design	14
3.1 Requirements	14
3.1.1 Functional Requirements	14
3.1.2 Non-Functional Requirements	15
3.1.3 Hardware and Software Requirements	16
3.2 Proposed Methodology	16
3.2.1 Overall Workflow	16
3.3 System Architecture.....	18
3.3.1 Main Parts of the System	18
3.3.2 How Data Moves in the System.....	20
3.3.3 Architecture Conclusion	20
3.4 Use Cases.....	21
3.4.1 Ask Legal Question.....	21
3.4.2 Upload Document for OCR	22
3.4.3 Generate Legal Document	22
3.4.4 Register / Login.....	23
3.4.5 Get Lawyer Recommendation (Lawyer Connect Module).....	24
3.4.6 View Legal Guidance	24
3.4.7 Manage Users.....	25
3.4.8 Manage Legal Content.....	25
3.4.9 Monitor System Activity.....	26
3.4.10 View User Queries	26
3.4.11 Accept / Reject Requests (Lawyer Connect Module).....	27
3.5 Methodology Diagram	28
3.6 Use Case Diagram.....	29
3.7 Architecture Diagram.....	30
3.8 System Pipeline Diagram.....	30
3.9 Database Design.....	30
3.10 Sequence diagram	31
3.11 GUI Graphical User Interfaces	31
Chapter 4: Implementation and Test Cases	36
4.1 Implementation	36

4.1.1 User Interface Implementation (Current Status).....	36
4.1.2 Implementation of Chat-Bot Module.....	37
4.1.3 Implementation of RAG on Chat-Bot.....	45
4.2 Test case Design and description.....	48
4.2.1 Sample Test case No.1	48
4.2.2 Sample Test case No.2	49
4.3 Test Metrics	49
4.3.1 Sample Test case Metric.No.1	49
4.3.2 Sample Test case Metric.No.2	49
4.3.3 Sample Test case Metric.No.3	49
Chapter 5: Experimental Results and Analysis.....	50
Chapter 6: Conclusion and Future Directions.....	51
References.....	52
Appendix.....	53
Appendix A: Guidelines	53
Appendix B: Heading of Sample Appendix B.....	53
Formatting Guidelines	54
Chapter 1: Heading 1	54
1.1 Heading 2.....	54
1.1.1 Heading 3.....	54
Tables and Figures	55
Equations.....	56
Header/Footer	56
Other Formatting Guidelines	56
References.....	56

List of Tables

Table 2.1:Litreture Review Summary Table	10
Table 2.2: Market Survey Summary	12
Table 3.1: U.C 1.....	21
Table 3.2: U.C 2.....	22
Table 3.3: U.C 3.....	22
Table 3.4: U.C 4.....	23
Table 3.5: U.C 5.....	24
Table 3.6: U.C 6.....	24
Table 3.7: U.C 7.....	25
Table 3.8: U.C 8.....	25
Table 3.9: U.C 9.....	26
Table 3.10: U.C 10.....	26
Table 3.11: U.C 11.....	27
Table 4.1: Datasets Used.....	42
Table 1.1: This is Sample table caption	55
Table 1.2: This is Sample table caption	55

List of Figures

Figure 3.1: Methodology	28
Figure 3.2: Use cases	29
Figure 3.3: Architecture Diagram	30
Figure 3.4: System Pipe-Line	30
Figure 3.5: ERD	31
Figure 3.6: Sequence Diagram.....	31
Figure 3.7: GUI 1.....	32
Figure 3.8: GUI 2.....	33
Figure 3.9: GUI 3.....	34
Figure 3.10: GUI 4.....	35
Figure 4.1: Training and Validation Loss	44
Figure 1.1: List of Styles.....	55
Figure 1.2: IEEE Reference style.....	56

Abstract

Legal information is typically written in technical language and is primarily intended for legal professionals, making it difficult for common citizens to understand their rights and legal procedures. This project proposes **Legal Mate**, an AI-based legal assistance system aimed at providing simplified and accessible legal guidance, with an initial focus on **property law**. The system is implemented as a mobile application integrated with an AI chatbot powered by a fine-tuned large language model. A domain-specific dataset was collected from multiple legal sources, cleaned, converted into instruction format, and augmented using synthetic data generation techniques. The chatbot was fine-tuned using the QLoRA approach to efficiently adapt the model on limited hardware resources. Validation-based evaluation during fine-tuning demonstrated stable training behavior and legally relevant responses for property-related queries. In its current phase, the project successfully implements the chatbot module and basic user interface, while full system testing and additional features are planned for future development. The results indicate that AI-driven systems can effectively assist citizens by simplifying legal information without replacing professional legal services.

Chapter 1: Introduction

Due to the complexity of the language, access to legal information remains a significant challenge for the citizens of Pakistan. High consultation costs, and limited availability of legal professionals also plays a crucial role in this. Most people find it difficult to understand their rights, obligations and procedures without professional assistance. This problem becomes more severe for people from non-legal backgrounds, especially when dealing with court procedures and legal documentations etc.

To close this gap, **Legal Mate**, an AI-based legal aid system, is suggested. Through an intelligent conversational interface, the system seeks to give users automatic document generation, basic legal counsel, and simplified explanations of legal topics. Legal Mate aims to improve the general public's access to, comprehension of, and affordability of legal information by utilizing massive language models, NLP techniques, and legal knowledge sources.

The goal of this project is to create an intelligent legal chatbot that can comprehend user enquiries written in common language, recognize what the user is asking, and offer relevant legal answers. The system supports many input methods, including text, voice, and optical character recognition (OCR), to make it more accessible and user-friendly.

1.1 Goals and Objectives

The primary **goal** of the Legal Mate project is to create an intelligent legal assistant that uses AI-driven technology to make it easier to obtain legal advice and information.

The main **objectives** of this project are listed below:

- To build an AI-based chatbot that can understand legal questions using Natural Language Processing (NLP).
- To explain legal terms, laws, and procedures in simple language so that people without a legal background can understand them easily.
- To create a conversational system that can give relevant and meaningful responses based on user questions.
- To add a feature that automatically generates basic legal documents such as applications, affidavits, and agreements.

Introduction

- To allow users to interact with the system using different input methods, including text, voice, and images through OCR.
- To connect the chatbot with a legal knowledge base that includes laws, rules, and past legal cases.
- To test and evaluate the system based on how accurate, useful, and easy to use it is.
- To help users get basic legal guidance without always needing to consult a lawyer immediately.
- To provide a lawyer connect feature that allows users to directly contact registered lawyers through the application for further legal guidance.

1.2 Scope of the Project

Legal Mate's focuses to give users broad legal information and advice. Initially, the scope of this system is limited to handling legal queries related to **property disputes**, allowing the system to focus on a specific legal domain and provide more accurate and relevant guidance. Its goal is to make legal concepts, rights, and procedures easier for people to understand. The system does not represent users in court or provide legally binding rulings, nor is it intended to take the place of expert legal counsel. The system also includes a lawyer connect module where registered lawyers can receive legal queries from users and choose to accept or reject them. Upon acceptance, the lawyer's contact details are shared with the user for further communication.

The project includes:

- Development of an AI-based legal chatbot using a large language model trained on **property**-related data.
- Answering general legal questions related to property laws, rights, penalties, and legal procedures.
- Automatically generating basic legal documents for guidance purposes.
- Providing multilingual support where possible to make the system more accessible.
- Using NLP techniques to understand user intent and generate suitable responses.
- Designing an easy-to-use and interactive interface for users.

The project does not include:

- Representing users in court or handling litigation cases.

- Making real-time legal decisions or predicting court judgments.
- Replacing qualified lawyers or legal professionals.
- Processing highly sensitive or confidential legal information.

By clearly defining these boundaries, the project ensures that Legal Mate is used responsibly and ethically, while still being a reliable and helpful tool for users seeking basic legal information.

Chapter 2: Literature Review

2.1 Introduction

This chapter presents a review of existing research and related work relevant to the development of the Legal Mate system. The purpose of this literature review is to understand how similar problems have been addressed in the past, what methods and technologies have been used, and what limitations still exist in current solutions. Reviewing previous studies helps in identifying suitable approaches and avoiding duplication of work while building upon existing knowledge.

2.2 Background and Problem Elaboration

Legal systems are built on complex rules, formal language, and detailed documentation, which makes legal information difficult to understand for people without a legal background. Legal documents often contain technical terms, long sentences, and references to laws and precedents that are not easily understandable by the general public. As a result, individuals facing legal issues frequently struggle to identify their rights, responsibilities, and the correct legal procedures.

Access to legal assistance is also limited by time, cost, and availability of legal professionals. Consulting a lawyer for basic legal guidance can be expensive and is not always immediately available, especially for individuals dealing with common disputes such as property-related matters. This creates a gap where users need quick and reliable legal information but are unable to obtain it easily through traditional legal services.

With the advancement of artificial intelligence (AI) and natural language processing (NLP), researchers have explored automated systems that can assist users by answering legal questions and processing legal text. Legal question answering systems aim to understand user queries written in natural language and retrieve or generate relevant legal responses. However, due to the complexity of legal language and the wide variety of legal domains, developing accurate and reliable legal assistance systems remains a challenging task.

Existing legal AI systems often suffer from several limitations. Many systems are rule-based, making them inflexible and difficult to scale to new legal domains. Others rely on large datasets that may not cover specific areas such as property disputes or local legal frameworks. In addition, many research systems focus on theoretical models and lack practical deployment in user-friendly platforms such as mobile applications. Limited support for multilingual input, voice-based interaction, and document understanding through OCR further reduces accessibility for a wide range of users.

These challenges highlight the need for an intelligent, accessible, and domain-focused legal assistance system. There is a clear requirement for a solution that can understand user queries in simple language, provide clear and relevant legal guidance, and support modern interaction methods. Addressing these issues forms the foundation for the development of the Legal Mate system, which aims to provide simplified legal assistance with an initial focus on property-related disputes.

2.3 Detailed Literature Review

2.3.1 Definitions

- **Artificial Intelligence (AI)**

Artificial Intelligence refers to computer systems designed to perform tasks that normally require human intelligence, such as reasoning and decision-making. In the legal field, AI is used to analyse legal text and assist users in understanding legal information.

- **Natural Language Processing (NLP)**

Natural Language Processing is a branch of AI that enables machines to understand and process human language. NLP techniques help legal systems analyse user queries and interpret complex legal language accurately.

- **Question Answering (QA) Systems**

Question Answering systems are designed to provide direct answers to questions asked in natural language. These systems use NLP and machine learning techniques to search information and return relevant responses.

- **Legal Question Answering (LQA)**

Legal Question Answering focuses on answering questions related to laws, regulations, and legal documents. LQA systems require high accuracy due to the sensitive and complex nature of legal information.

- **Deep Learning**

Deep learning is a machine learning approach that uses multi-layer neural networks to learn complex patterns from data. It is widely used in legal QA systems to improve understanding of legal language and context.

- **Large Language Models (LLMs)**

Large Language Models are advanced deep learning models trained on large text datasets to understand and generate human-like language. In legal applications, LLMs help produce more accurate and context-aware responses.

2.3.2 Related Research Work 1

Answering questions related to the legal domain is a complex task due to the intricate nature of legal language and the wide variety of legal document systems [1]. The authors explain that providing accurate answers to legal questions often requires specialized legal knowledge, which makes the task challenging even for human experts. To address this challenge, the study focuses on legal question answering (LQA) systems, which aim to generate answers to legal questions expressed in natural language using artificial intelligence and natural language processing techniques [1].

The study presents a comprehensive survey of existing legal question answering research rather than proposing a single implemented system. The authors review fourteen benchmark datasets used for legal QA and analyse state-of-the-art deep learning models applied in this domain [1]. According to the paper, legal QA systems typically use natural language processing to understand the meaning of a question and then apply machine learning and information retrieval techniques to locate the most relevant answers from large collections of legal documents. The survey highlights the growing use of deep learning and transformer-based

architectures due to their ability to handle complex and unstructured legal text and to capture contextual information more effectively than traditional rule-based approaches [1].

The findings of the survey show that deep learning-based legal QA systems have achieved improved performance across different datasets and evaluation metrics [1]. The authors report that these models benefit from end-to-end learning and the ability to learn contextual relationships within legal text. However, the study also identifies several limitations, including the need for large amounts of high-quality labelled data, domain-specific datasets, and challenges related to reliability, transparency, and trustworthiness of legal QA systems. Ethical concerns, data privacy issues, and the risk of incorrect or misleading answers are also highlighted as major challenges for real-world deployment [1].

In conclusion, this survey provides a detailed overview of existing legal question answering systems, datasets, and deep learning techniques, highlighting both their strengths and limitations [1]. While the reviewed approaches demonstrate the effectiveness of deep learning models in addressing legal QA challenges, the study indicates that many existing systems remain research-oriented and lack practical, user-friendly deployment. These limitations justify the need for applied systems such as Legal Mate, which aim to utilize legal QA techniques in a more accessible and domain-focused manner, particularly for property-related legal disputes.

2.3.3 Related Research Work 2

The research work titled “Legal Document Generation: Harnessing Large Language Models for Diverse Legal Workflows” addresses the challenge of automating the creation of legal and process-related documents using Large Language Models (LLMs). The authors explain that preparing legal documents such as Statements of Work, procurement documents, treasury reports, and policy documents is a labour-intensive and time-consuming process that often requires weeks of manual effort. Although LLMs have been applied in multiple domains, the paper highlights that their use for structured and compliant legal document generation remains limited, creating a gap in existing legal automation solutions [2].

To bridge this gap, the authors propose an automated legal document generation framework that integrates LLMs with prompt engineering and a multi-agent system. The methodology is based on predefined legal templates created with the assistance of legal experts to ensure accuracy and compliance. Multiple specialized agents are used for different tasks, including

information extraction from user input, semantic identification, template analysis, and content generation. The system also applies section-specific prompt refinement to ensure that each part of the document follows the required legal structure. In addition, Python libraries are used to generate tables and flowcharts, particularly for process-related legal documents [2].

The results of the study demonstrate a significant improvement in efficiency when compared to manual document preparation. Through time and motion studies, the authors show that documents which traditionally took several hours or days to create can now be generated within minutes. The automated approach also reduces errors, improves consistency, and ensures compliance with legal standards. However, the study identifies limitations such as dependency on predefined templates, document-specific customization requirements, and language constraints, as the system primarily supports English-language documents [2].

In conclusion, this research presents a structured and practical approach to automated legal document generation using LLMs, prompt engineering, and multi-agent systems. While the proposed solution significantly improves efficiency and accuracy, its reliance on templates and document-specific configurations highlights the need for more flexible and user-oriented systems. These findings support the motivation behind Legal Mate, which aims to incorporate automated document generation features in a simplified and accessible manner for property-related legal use cases.

2.3.4 Related Research Work 3

This research proposes an AI Legal Assistant aimed at providing preliminary legal guidance by analysing user-submitted information related to legal and criminal issues. The system allows users to submit queries using multiple input formats, including text, voice, images, and videos. Based on the provided information, the system identifies relevant crime categories, applicable Indian Penal Code (IPC) sections, punishment details, and related case laws. The study clearly states that the system is intended only for initial legal awareness and does not replace professional legal advice [3].

The proposed solution follows a modular architecture that includes a user interface, a Flask-based backend server, a multimodal AI model, and a database. User inputs are processed according to their type, where voice inputs are converted into text and multimedia inputs are

prepared for analysis by the AI model. The backend server manages communication between the interface and the AI model to ensure structured and controlled legal outputs [3].

A notable contribution of this work is the use of a multimodal large language model, which enables the system to analyse not only text but also multimedia evidence. The system generates structured legal outputs such as crime classification, IPC sections, punishment details, and related case laws. The results are stored in a database and presented to users in both text and voice formats, which improves accessibility, especially for users facing language barriers [3].

While the system demonstrates accurate identification of legal sections and classifications, it has several limitations when considered from a broader legal assistance perspective. The system is primarily focused on criminal law and IPC sections, making it less suitable for other legal domains such as property disputes. Additionally, the system does not support automated legal document generation or controlled interaction with legal professionals. These limitations are addressed in the proposed Legal Mate system by focusing on property-related legal guidance, supporting legal document generation, and introducing a lawyer connect module that allows users to directly interact with registered lawyers in a controlled manner. Furthermore, Legal Mate is designed as a mobile application, improving accessibility and practical usability compared to the web-based approach discussed in this study [3].

2.3.5 Related Research Work 4

This research presents LawPal, a retrieval-augmented generation (RAG) based legal chatbot developed to improve legal accessibility in India. The system aims to reduce misinformation and improve public understanding of legal matters by answering user queries using authoritative legal sources such as statutory laws, court judgments, and the Indian Constitution. Unlike traditional keyword-based chatbots, LawPal combines document retrieval with generative AI to provide more accurate and context-aware legal responses [4].

The proposed system uses a Retrieval-Augmented Generation (RAG) approach, where legal documents are first collected and pre-processed, then converted into vector embeddings using the DeepSeek-R1:5B model. These embeddings are stored and indexed using FAISS (Facebook AI Similarity Search) to enable fast and relevant document retrieval. When a user submits a legal query, the system retrieves the most relevant legal text chunks and provides

them as context to the language model before generating a response. This approach helps reduce hallucinations and ensures responses remain grounded in verified legal sources [4].

The authors report that LawPal achieves high accuracy and efficiency in legal information retrieval. Evaluation results show over 90% legal accuracy, fast response times, and strong performance when handling complex or ambiguous legal questions. The system also outperforms rule-based and keyword-based legal chatbots in terms of relevance and contextual understanding. However, the study highlights that LawPal is primarily designed for legal research and information access, rather than practical legal assistance workflows [4].

Despite its strengths, LawPal has certain limitations when viewed from an applied legal support perspective. The system mainly focuses on answering legal queries and does not provide automated legal document generation or direct interaction with legal professionals. Additionally, the chatbot is implemented as a web-based system and is centred on general legal knowledge rather than domain-specific use cases such as property disputes. These limitations are addressed in the proposed Legal Mate system by offering property-focused legal guidance, legal document generation, and a lawyer connect module that allows users to directly contact registered lawyers in a controlled manner. Furthermore, Legal Mate is designed as a mobile application, improving accessibility and real-world usability compared to the research-oriented design of LawPal [4].

2.3.6 Related Research Work 5

This research introduces VLegal-Bench, a comprehensive benchmark designed to evaluate the legal reasoning capabilities of large language models in the context of Vietnamese law. The study highlights that existing legal benchmark mainly focus on English or Chinese legal systems and are not suitable for civil law jurisdictions like Vietnam, where laws are highly codified, hierarchical, and frequently updated. The benchmark is designed to assess how well language models understand, retrieve, and reason over legal texts in real-world legal assistant scenarios [5].

The proposed benchmark is built using a cognitively grounded framework inspired by Bloom's taxonomy, covering five levels of legal understanding: recognition and recall, understanding and structuring, reasoning and inference, interpretation and generation, and ethics and fairness. VLegal-Bench contains 10,450 expert-verified samples collected from authoritative

Vietnamese legal documents and real citizen legal questions. These samples are designed to simulate realistic legal assistant tasks such as legal question answering, retrieval-augmented generation, multi-step reasoning, and scenario-based legal analysis [5].

Extensive experiments were conducted on a wide range of general-purpose and domain-adapted language models. The results show that while many models perform well on basic legal recall tasks, their performance drops significantly when dealing with complex legal reasoning, conflict detection between legal provisions, and multi-article inference. The study also demonstrates that domain-adapted legal models outperform large general-purpose models on higher-level legal reasoning tasks, emphasizing the importance of legal-specific training and evaluation [5].

Although VLegal-Bench provides a strong evaluation framework, it focuses only on benchmarking and evaluation, rather than delivering a deployable legal assistance system. The benchmark does not provide user-facing features such as legal document generation, mobile accessibility, or direct interaction with legal professionals. Additionally, it is specific to Vietnamese law and is not directly usable by general users seeking legal guidance. These limitations are addressed in the proposed Legal Mate system by offering a practical mobile application, AI-based legal assistance, automated document generation, and a lawyer connect module, making the system more suitable for real-world legal support rather than academic evaluation alone [5].

2.4 Literature Review Summary Table

The summary of the reviewed literature related to AI-based legal question answering and automated legal document generation is presented in Table 2. This table highlights the focus, techniques used, and limitations of existing research works in comparison to the proposed Legal Mate system.

Table 2.1: Literature Review Summary Table

No.	Paper Title	Year	Focus / Domain	Method Used	Tools / Models	Key Results	Limitations
-----	-------------	------	----------------	-------------	----------------	-------------	-------------

1	Exploring the State of the Art in Legal QA Systems	2023	Legal Question Answering	NLP, Deep Learning, Transformer-based QA models	Legal QA datasets, Deep Learning models	Improved accuracy in answering legal questions using deep learning approaches	Research-focused, lacks deployable system, no domain-specific focus, no document generation or lawyer interaction
2	Legal Document Generation: Harnessing LLMs for Diverse Legal Workflows	2024	Legal Document Automation	LLMs, Prompt Engineering, Multi-Agent System, Templates	Large Language Models, Python libraries	Significant reduction in document preparation time and improved consistency	Template dependency, limited flexibility, mainly enterprise-focused, not user-centric
3	AI Legal Assistant	2025	Criminal Law (IPC-based)	Multimodal AI, NLP, Rule-based structuring	Multimodal LLM, Flask backend	Accurate identification of IPC sections and punishments, improved accessibility	Limited to criminal law, no document generation, no lawyer interaction, web-based only
4	LawPal: RAG-Based System for Legal Accessibility	2025	General Legal Information	Retrieval-Augmented Generation (RAG)	DeepSeek-R1:5B, FAISS	High legal accuracy and reduced hallucinations in responses	No document generation, no lawyer connect feature, research-oriented, web-based
5	VLegal-Bench: Benchmark for Legal Reasoning	2025	Legal Reasoning Evaluation	Benchmarking, Cognitive Taxonomy	Legal datasets, LLM evaluation frameworks	Identified weaknesses of LLMs in complex legal reasoning	Evaluation-only, not a deployable system, country-specific, no user-facing features

2.5 Market Survey

Table 2.2: Market Survey Summary

Project / Org	Target users	Lawyer connect	Retrieval-Augmented Generation (RAG) / statute-aware answers	Urdu & Roman-Urdu support	Voice & OCR inputs	Document generation (fillable templates)	Provenance & statutory citations shown
VakeelOnline	General public (client ↔ lawyers)	Yes, primary feature is connecting clients to lawyers.	No (search/marketplace)	Limited (app focused on case matching, language not primary claim)	No	No	No (no evidence of per-answer statute provenance)
Pakistan Law Bot	General public / legal professionals / law students	Some contact/referral features likely	AI chatbot but RAG not used.	No, only translates output	No / OCR-only paid version	Yes, but limited	No
Pak Legal Database (PLD)	Legal professionals / researchers	No (only research tool, not client marketplace)	Yes, Advanced search & research but focused on professional research, not public simplified RAG.	Primarily English/No roman - uses in built google translator	No	Only offer downloadable docs for professionals, not citizen-ready templates.	Yes (provides primary sources/judgments) but not simplified citizen-facing answers.
Pakistan Law Site (repositories)	Researchers, students, lawyers	No	No (Only a legal text repositories)	Primarily English/legal language	No	No	Yes (source texts) but not presented as simplified answers.

2.6 Research Gap

The reviewed literature shows that strong progress has been made in legal question answering and automated legal document generation using artificial intelligence and natural language processing. Techniques based on deep learning and large language models are effective in understanding legal text and producing structured outputs. Despite these advances, several important gaps remain when these systems are considered for real-world use.

Most existing legal question answering systems are developed mainly for research purposes and are built on general legal datasets. These systems often lack user-friendly and deployable solutions for everyday users. In addition, very limited work addresses controlled lawyer-client interaction, where lawyers have the ability to accept or reject user requests before any direct contact takes place. Many systems also do not focus on specific legal domains, such as property disputes, even though these issues are common and highly relevant to the general public. Furthermore, many studies emphasize model performance in experimental settings rather than practical usability, accessibility, and mobile-based deployment.

Similarly, current automated legal document generation systems rely heavily on predefined templates and are mostly designed for organizational or enterprise use. Such systems often require technical expertise, structured input, or controlled environments, which makes them less suitable for direct use by ordinary users. Limited support for multimodal inputs, such as voice queries and document images, further reduces accessibility.

These limitations indicate the need for an intelligent, easy-to-use, and domain-focused legal assistance system that integrates legal question answering, document generation, and controlled lawyer interaction within a single mobile platform. Addressing these gaps provides the motivation for the development of the Legal Mate system.

2.7 Problem Statement

Despite progress in artificial intelligence and natural language processing, many people still struggle to access legal information that is clear and easy to understand. Most existing legal assistance systems are either designed for research purposes or built for enterprise use, which makes them difficult for ordinary users to access and use. As a result, individuals without legal knowledge often face challenges in understanding their rights and legal procedures.

Currently, there is no widely available mobile-based solution that offers simplified legal guidance along with automated document generation and support for modern input methods such as voice and document images. This limitation is especially noticeable in property-related legal disputes, where people frequently require basic legal help but cannot easily consult a lawyer. Therefore, there is a need to develop an intelligent and user-friendly legal assistance system that provides reliable, accessible, and domain-focused legal support through a mobile application.

Chapter 3: Requirements and Design

The Legal Mate system is designed as a mobile application that provides legal guidance related to property disputes using artificial intelligence. The system is planned by first understanding how users interact with the application and how different technical components work together in the background. The focus of the design is to keep the user experience simple while handling complex tasks such as AI-based legal responses, document generation, and data storage efficiently.

The application supports multiple ways of interaction, including text, voice, and document images, so users can choose the most convenient method. The backend, AI services, and databases are connected in a structured way to ensure smooth data flow and reliable system behaviour. The requirements and design described here form the base for implementing the Legal Mate system in a practical and reproducible manner.

3.1 Requirements

The requirements of the Legal Mate system are defined based on user needs and expected system behaviour. These requirements ensure that the system performs its tasks correctly while remaining secure, reliable, and easy to use. The requirements are grouped into functional, non-functional, and technical requirements for clarity.

3.1.1 Functional Requirements

The Legal Mate system must provide the following core functionalities:

FR.1: The system shall allow users to register and log in securely.

FR.2: The system shall restrict access to features until the user is authenticated.

FR.3: The system shall allow users to ask property-related legal questions using text input.

FR.4: The system shall support voice input by converting speech into text.

FR.5: The system shall allow users to upload images of legal or property documents.

FR.6: The system shall extract text from uploaded images using OCR.

FR.7: The system shall process user queries using an AI-based chatbot.

FR.8: The system shall provide legal guidance focused on property disputes.

FR.9: The system shall generate basic legal documents such as applications, notices, and affidavits.

FR.10: The system shall generate documents in DOCX format.

FR.11: The system shall allow users to download generated documents.

FR.12: The system shall store user queries and generated documents for future reference.

FR.13: The system shall recommend lawyers when required.

FR.14: The system shall allow the admin to manage users and monitor system activity.

FR.15: The system shall allow lawyers to register and create a professional profile.

FR.16: The system shall allow users to view a list of registered lawyers.

FR.17: The system shall allow users to select a specific lawyer and submit a legal query.

FR.18: The system shall notify the selected lawyer of the received query.

FR.19: The system shall allow lawyers to accept or reject user requests.

FR.20: The system shall share the lawyer's contact details with the user only after acceptance.

3.1.2 Non-Functional Requirements

In addition to system features, Legal Mate is designed to meet certain quality and performance expectations. The application is kept simple and easy to use so that users without legal knowledge can understand the responses. The system responds to user queries within a reasonable time to maintain a smooth user experience.

Requirements and Design

User data is protected through secure authentication and controlled access. The system is designed to be reliable and available for regular use without frequent failures. Scalability is also considered so that additional legal domains or features can be added in the future without major changes. Accuracy and consistency in legal responses are maintained to ensure user trust in the system.

3.1.3 Hardware and Software Requirements

The system requires the following hardware and software components to operate:

3.1.3.1 Hardware Requirements

- A smartphone (Android or iOS)
- Stable internet connection
- Server or cloud infrastructure for backend services

3.1.3.2 Software Requirements

- Flutter and Dart for mobile application development
- Node.js for backend API services
- Python for AI chatbot and processing services
- MongoDB for storing user data and system records
- Pinecone for vector storage to support retrieval-augmented generation
- OCR engine for extracting text from document images
- Speech-to-text service for voice input

3.2 Proposed Methodology

The Legal Mate system follows a simple and structured workflow that allows users to receive property-related legal guidance through a mobile application. The system is designed so that users can interact naturally, while all complex processing happens in the background without requiring technical knowledge.

3.2.1 Overall Workflow

The working of the Legal Mate system can be described in the following steps:

1. User Authentication

The process starts when a user logs into the Legal Mate mobile application. Only authenticated users are allowed to access system features to ensure data security and controlled access.

2. User Input Submission

After logging in, the user can submit legal queries using one of the following methods:

- Text input through the chatbot interface
- Voice input, which is converted into text
- Image upload of legal or property documents for OCR processing

3. Input Processing

- Text queries are sent directly to the backend server.
- Voice input is first converted into text using a speech-to-text service.
- Uploaded document images are processed using OCR to extract readable text.

4. AI-Based Analysis

The processed text is forwarded from the Node.js backend to a Python-based AI service. A locally hosted large language model, fine-tuned on legal data, analyses the input. To improve accuracy and to provide the correct citations, relevant legal information is retrieved from a vector database and used as additional context.

5. Response Generation

Based on the analysis, the AI model generates a legal response focused on property disputes. The response is returned to the backend and displayed to the user in simple and understandable language.

6. Document Generation (If requested)

When the user requests a legal document:

- Required details are collected through the application

Requirements and Design

- Predefined templates are combined with AI-generated content
- The document is created in DOCX format
- The user is allowed to download the document

7. Data Storage

User queries, generated responses, and documents are stored in the database for future reference and system improvement.

8. Lawyer Connect Workflow

- The user opens the lawyer connect module and selects a lawyer.
- The user submits a legal query to the selected lawyer.
- The query is sent to the backend and stored.
- The lawyer receives the request and reviews the query.
- The lawyer can either accept or reject the request.
- If accepted, the system shares the lawyer's phone number or email with the user.
- Further communication takes place outside the application.

3.3 System Architecture

The Legal Mate system is built using a simple and well-organized structure where each part of the system has a clear role. The main idea behind the architecture is to keep the user experience smooth while handling all complex work such as AI processing and data storage in the background. The system is divided into different parts so that changes can be made easily without affecting the entire application.

At a high level, the system consists of a mobile application, a backend server, AI services, and databases. These components communicate with each other to process user requests and return results.

3.3.1 Main Parts of the System

1. Mobile Application

The mobile application is the part of the system that users interact with. It allows users to:

- Create an account and log in

- Ask legal questions related to property disputes
- Use voice input or upload document images
- View legal responses from the AI
- Generate and download legal documents
- Browse registered lawyers and send queries to them.
- The mobile app sends all user actions to the backend server and displays the responses it receives.

2. Backend Server

The backend server acts as the main control unit of the system. It:

- Receives requests from the mobile app
- Checks user login and permissions
- Sends user queries to AI services
- Handles lawyer connect requests
- Stores and retrieves data from databases

The backend makes sure that data flows smoothly between different parts of the system.

3. AI Processing Service

The AI service is responsible for understanding legal questions and generating responses.

It runs as a separate Python-based service and includes:

- A locally hosted language model trained on legal data
- Logic to understand user queries and document text
- Support for retrieving relevant legal information from stored data

The AI service receives text from the backend and returns clear legal guidance focused on property disputes.

4. OCR and Voice Modules

The OCR module extracts text from images uploaded by users, such as property documents or legal papers. The voice module converts spoken queries into text. Once converted, the text is treated the same as typed input and sent to the AI service for processing.

5. Lawyer Connect Module

Requirements and Design

The lawyer connect module allows direct interaction between users and lawyers. In this module:

- Lawyers register and create their profiles in the system
- Users can view a list of available lawyers
- Users can select a lawyer and send a legal query
- Lawyers receive the query and can accept or reject it
- If accepted, the system shares the lawyer's contact details with the user

This approach ensures that lawyers remain in control and user privacy is maintained.

6. Databases

The system uses two types of databases:

- **MongoDB**, which stores user accounts, queries, lawyer profiles, and generated documents
- **Pinecone**, which stores legal information in vector form to support accurate AI responses

These databases help the system respond quickly and store information securely.

3.3.2 How Data Moves in the System

- The user performs an action in the mobile app
- The backend server receives the request
- Input is processed using OCR or voice services if needed
- The request is sent to the AI service for analysis
- The AI generates a response and sends it back
- The backend forwards the response to the mobile app
- If a lawyer accepts a request, contact details are shared with the user

3.3.3 Architecture Conclusion

The architecture of the Legal Mate system keeps all parts connected while maintaining simplicity and clarity. By separating user interaction, backend logic, AI processing, and data storage, the system remains easy to understand, flexible, and scalable. This design supports intelligent legal guidance and controlled lawyer interaction while allowing future improvements without major changes.

3.4 Use Cases

- **Use Cases for General User (Citizen)**
 1. Register Account
 2. Login to System
 3. Ask Legal Question (Chatbot)
 4. Upload Document for OCR
 5. Generate Legal Document
 6. Download Generated Document
 7. Connect with Lawyer
- **Use Cases for Lawyer**
 8. Register / Login as Lawyer
 9. Accept or Reject User Query
- **Use Cases for Admin**
 10. Manage Users, Lawyers and Data
 11. Monitor System Activity

3.4.1 Ask Legal Question

Table 3.1: U.C 1

Name	Ask Legal Question
Actors	General User
Summary	Allows a user to ask a legal question in text or voice and receive a statute-aware answer
Pre-Conditions	User is registered and logged in
Post-Conditions	Question is answered; query stored in database
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. User enters or speaks question	2. App sends request to backend
	3. Backend queries AI Chatbot (RAG-based)
	4. Chatbot returns answer

	5. Backend stores query and answer
	6. App displays answer to user
Alternative Flow	
1. Network failure	2. Show error message
3. Invalid question	4. Prompt for re-entry
5. AI timeout	6. Retry or fallback message

3.4.2 Upload Document for OCR

Table 3.2: U.C 2

Name	Upload Document for OCR
Actors	General User
Summary	Allows user to upload scanned documents (FIRs, notices, agreements) for AI-assisted interpretation
Pre-Conditions	User is logged in; document is in supported format
Post-Conditions	Document text extracted; legal summary displayed
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. User uploads document	2. App sends document to backend
	3. OCR module extracts text
	4. AI Chatbot processes extracted text
	5. Backend sends summarized interpretation
	6. App displays summary
Alternative Flow	
1. Unsupported document format	2. Show error
3. Poor scan quality	4. Prompt for re-upload

3.4.3 Generate Legal Document

Table 3.3: U.C 3

Name	Generate Legal Document
Actors	General User
Summary	Generates editable legal templates based on user input (FIR draft, affidavit, agreement)

Pre-Conditions	User is logged in; required fields available
Post-Conditions	Legal document generated and stored
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. User selects document type	2. App shows fillable form
3. User fills details	4. Backend Document Generation module creates document
	5. Document stored in database
	6. App displays downloadable document
Alternative Flow	
1. Missing required fields	2. Show warning
3. Backend error	4. Retry or show error message

3.4.4 Register / Login

Table 3.4: U.C 4

Name	Register / Login
Actors	General User
Summary	Allows user to create an account or log in to access LegalMate
Pre-Conditions	App installed; internet connectivity available
Post-Conditions	User authenticated and session started
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. User opens app	
2. User chooses Register or Login	
3. For registration: enter details	4. Backend validates, account created
5. For login: enter credentials	6. Backend authenticates
	7. App navigates to dashboard
Alternative Flow	
1. Invalid credentials	2. Prompt retry
3. Email already exists	4. Show error
5. Network failure	6. Show message

3.4.5 Get Lawyer Recommendation (Lawyer Connect Module)

Table 3.5: U.C 5

Name	Get Lawyer Recommendation
Actors	General User
Summary	Provides recommended lawyers based on specialization, rating, and location, and allows users to submit queries directly to them
Pre-Conditions	User logged in; lawyer database populated
Post-Conditions	Recommended lawyers displayed; request submitted to specific lawyer
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. User selects “Get Lawyer Recommendation.”	2. App sends request to backend
	3. Backend queries lawyer database and filters results
	4. Recommendations sent to app
5. User selects a specific lawyer and submits a query.	6. System sends query to the selected lawyer for review.
Alternative Flow	
1. No lawyers available	2. Display message
3. Network error	4. Retry or notify user

3.4.6 View Legal Guidance

Table 3.6: U.C 6

Name	View Legal Guidance
Actors	General User
Summary	Allows users to read simplified legal guidance in Urdu/Roman Urdu/English.
Pre-Conditions	User logged in
Post-Conditions	Guidance displayed; knowledge updated
Special Requirements	None
Basic Flow	
Actor Action	System Response

1. User selects “View Legal Guidance.”	2. Categories displayed (Property, Criminal, Cybercrime)
3. User selects topic	4. Backend retrieves guidance
	5. App displays in simplified language
Alternative Flow	
1. Topic not found	2. Notify user
3. Network failure	4. Retry

3.4.7 Manage Users

Table 3.7: U.C 7

Name	Manage Users
Actors	Admin
Summary	Admin can add, edit, or remove users from the system
Pre-Conditions	Admin logged in; proper permissions granted
Post-Conditions	User accounts updated in database
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. Admin selects “Manage Users.”	2. Views user list
3. Add, edit, or delete user	4. Backend updates database
	5. App confirms action
Alternative Flow	
1. Insufficient permissions	2. Deny action
3. Database error	4. Show message

3.4.8 Manage Legal Content

Table 3.8: U.C 8

Name	Manage Legal Content
Actors	Admin
Summary	Admin can add, update, or remove legal knowledge, templates, and guidance
Pre-Conditions	Admin logged in with proper privileges
Post-Conditions	Legal content updated in the system database
Special Requirements	None

Basic Flow	
Actor Action	System Response
1. Admin selects “Manage Legal Content.”	2. Views current knowledge base and templates
3. Adds, edits, or deletes content	4. Backend updates database
	5. Confirmation displayed to Admin
Alternative Flow	
1. Insufficient privileges	2. Deny action
3. Database error	4. Notify Admin

3.4.9 Monitor System Activity

Table 3.9: U.C 9

Name	Monitor System Activity
Actors	Admin
Summary	Admin monitors user queries, document generation, and AI interactions to ensure proper usage
Pre-Conditions	Admin logged in
Post-Conditions	Activity logs reviewed; alerts generated if needed
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. Admin opens “System Activity” dashboard	2. Reviews logs of user actions and backend processes
3. Identifies abnormal activities	4. Takes corrective action if required
Alternative Flow	
1. No logs available	2. Show message
3. Database failure	4. Retry

3.4.10 View User Queries

Table 3.10: U.C 10

Name	View User Queries
Actors	Lawyer

Summary	Lawyer can view legal questions submitted by users to provide professional advice
Pre-Conditions	Lawyer is registered and logged-in
Post-Conditions	Queries reviewed and available for response
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. Lawyer navigates to “User Queries.”	2. Backend fetches user-submitted questions.
	3. Queries displayed with timestamps and details
4. Lawyer marks as reviewed or responds	
Alternative Flow	
1. No queries	2. Display message
3. Network/database error	4. Retry

3.4.11 Accept / Reject Requests (Lawyer Connect Module)

Table 3.11: U.C 11

Name	Accept / Reject Requests
Actors	Lawyer
Summary	Lawyer reviews user queries and chooses to accept or reject the request for assistance
Pre-Conditions	Lawyer logged in; pending requests available
Post-Conditions	Request status updated; user receives contact info if accepted
Special Requirements	None
Basic Flow	
Actor Action	System Response
1. Lawyer navigates to “Pending Requests.”	2. Displays request details, query, and user info
3. Lawyer selects Accept or Reject	4. Backend updates request status
	5. If accepted, system sends lawyer’s phone/email to user
	6. User notified of decision

Alternative Flow	
1. Request already processed	2. Show message
3. Backend error	4. Retry action

3.5 Methodology Diagram

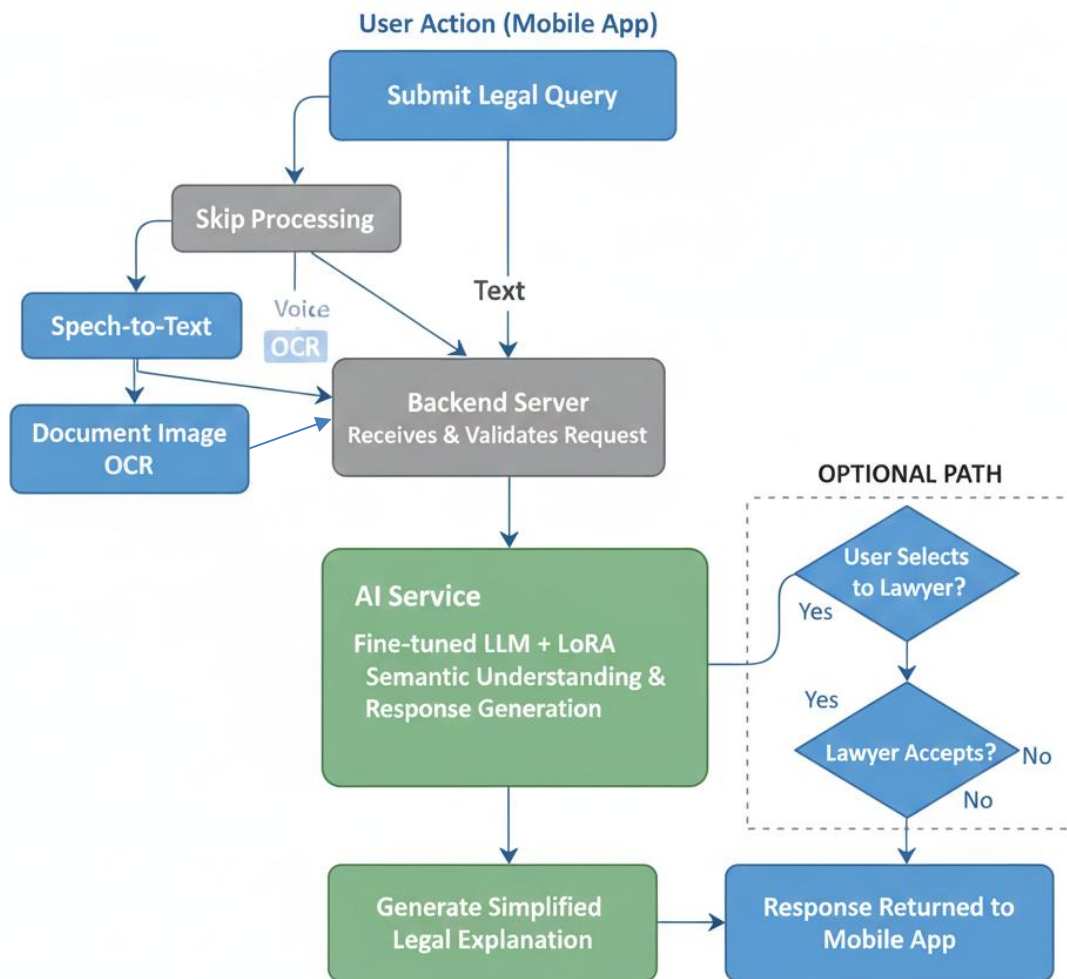


Figure 3.1: Methodology

3.6 Use Case Diagram

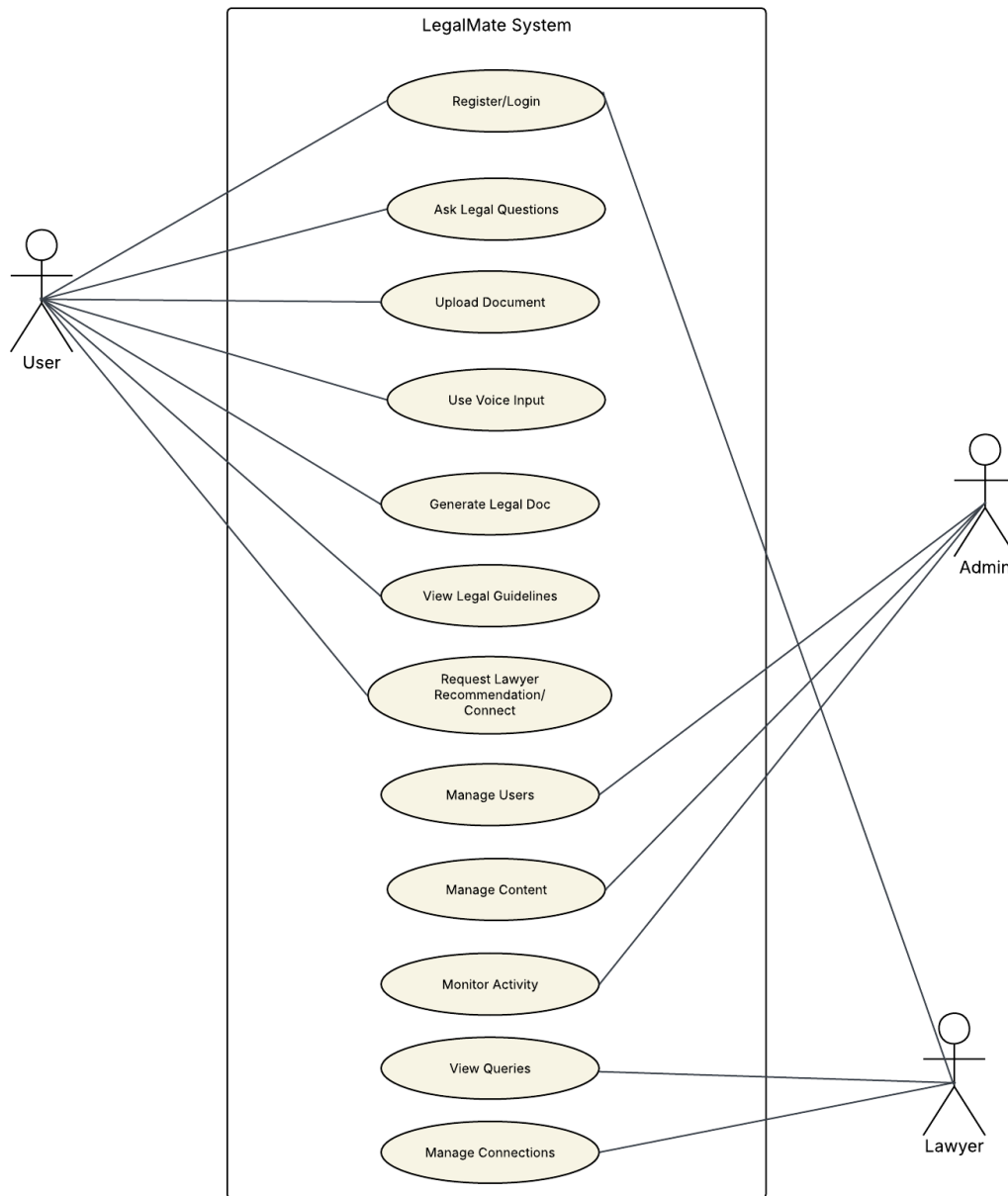


Figure 3.2: Use cases

3.7 Architecture Diagram

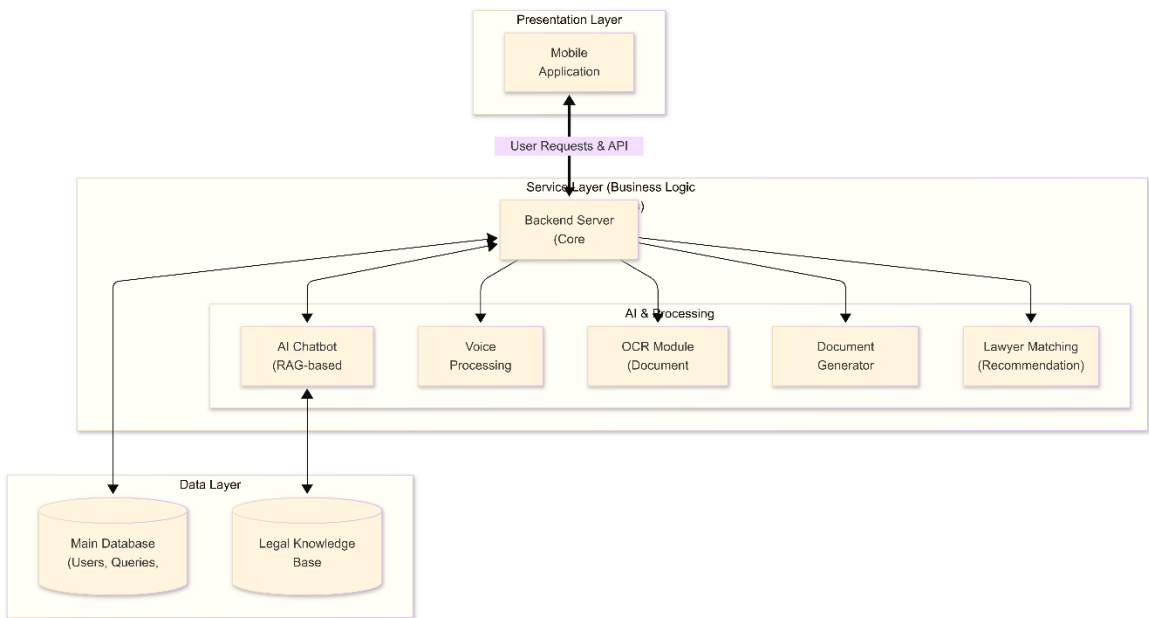


Figure 3.3: Architecture Diagram

3.8 System Pipeline Diagram

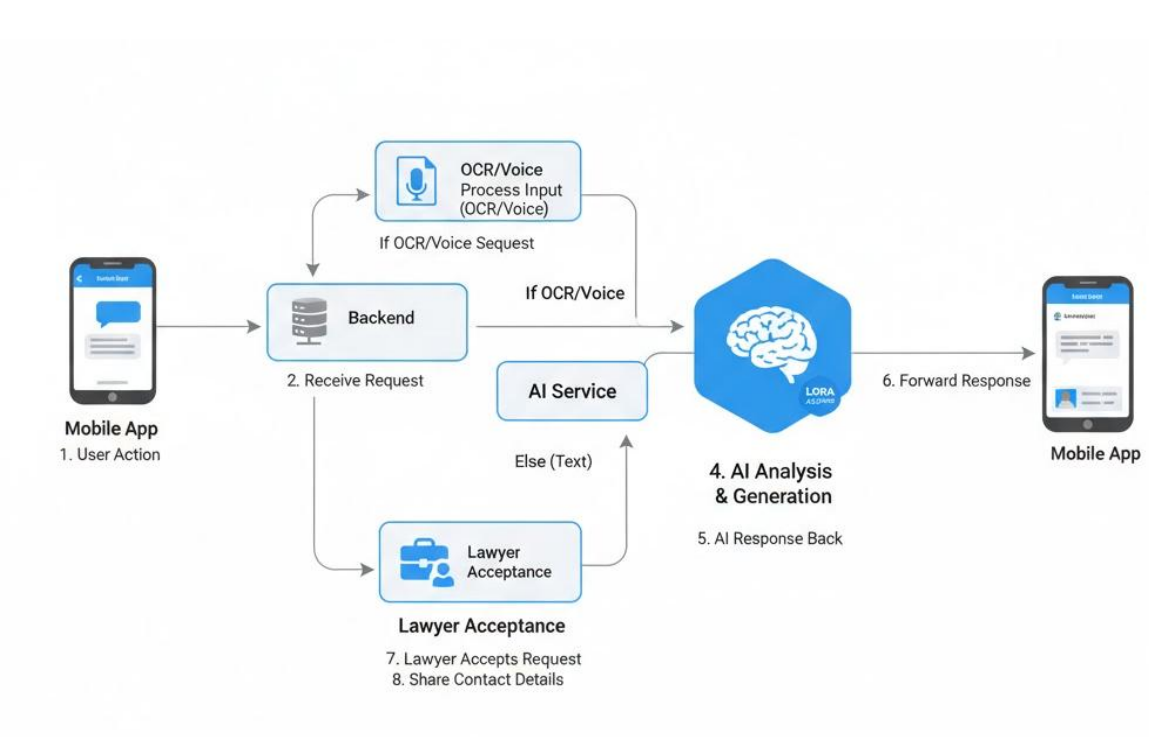


Figure 3.4: System Pipe-Line

3.9 Database Design

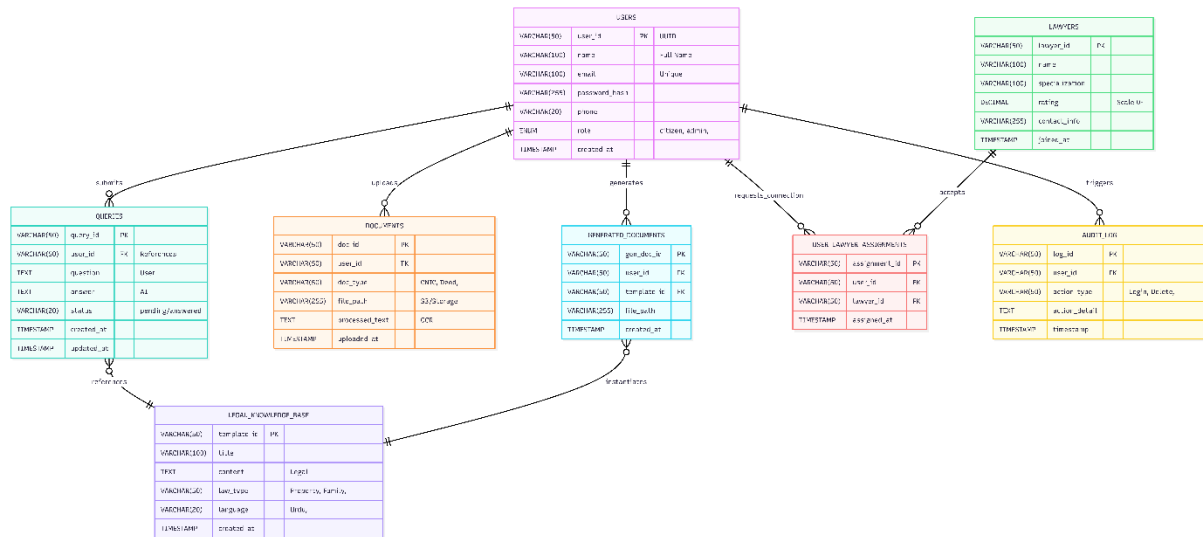


Figure 3.5: ERD

3.10 Sequence diagram

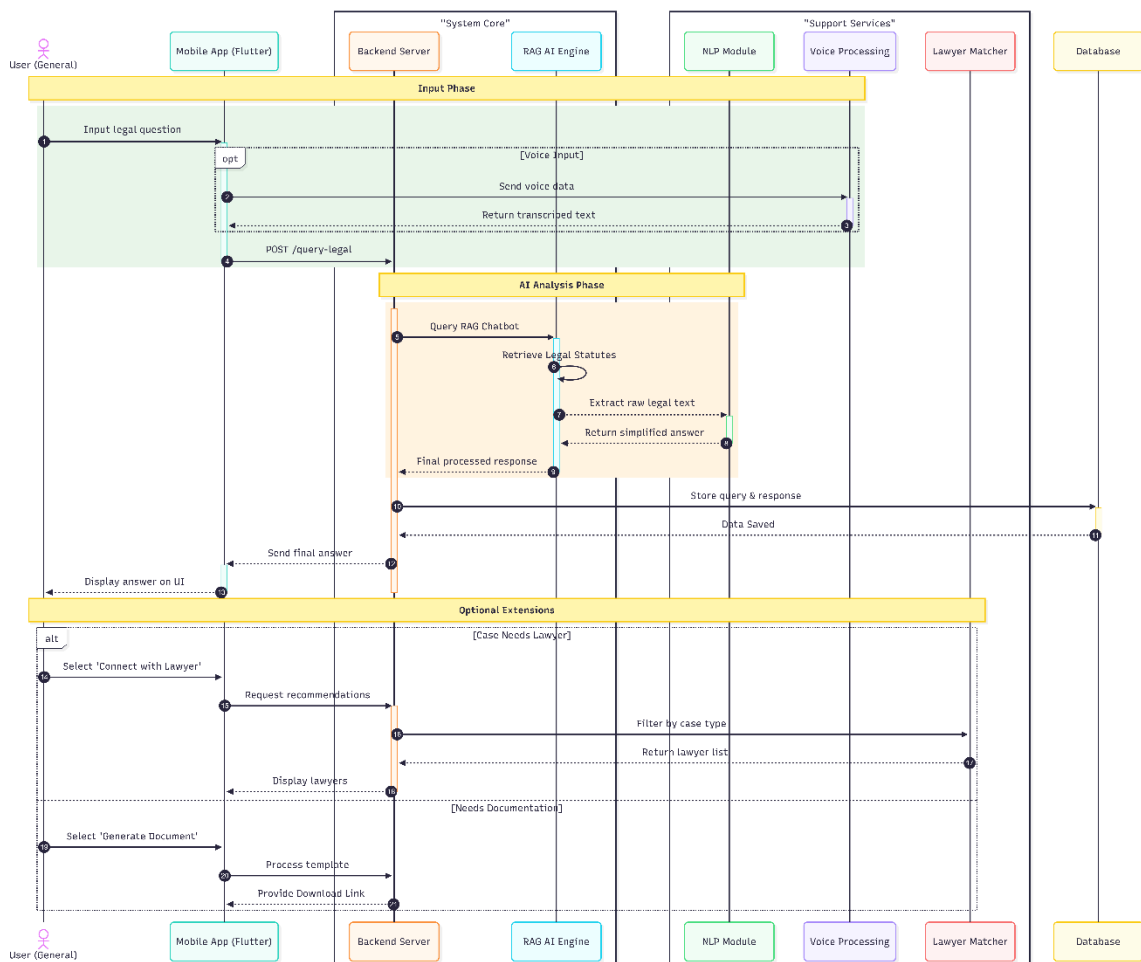


Figure 3.6: Sequence Diagram

3.11 GUI Graphical User Interfaces

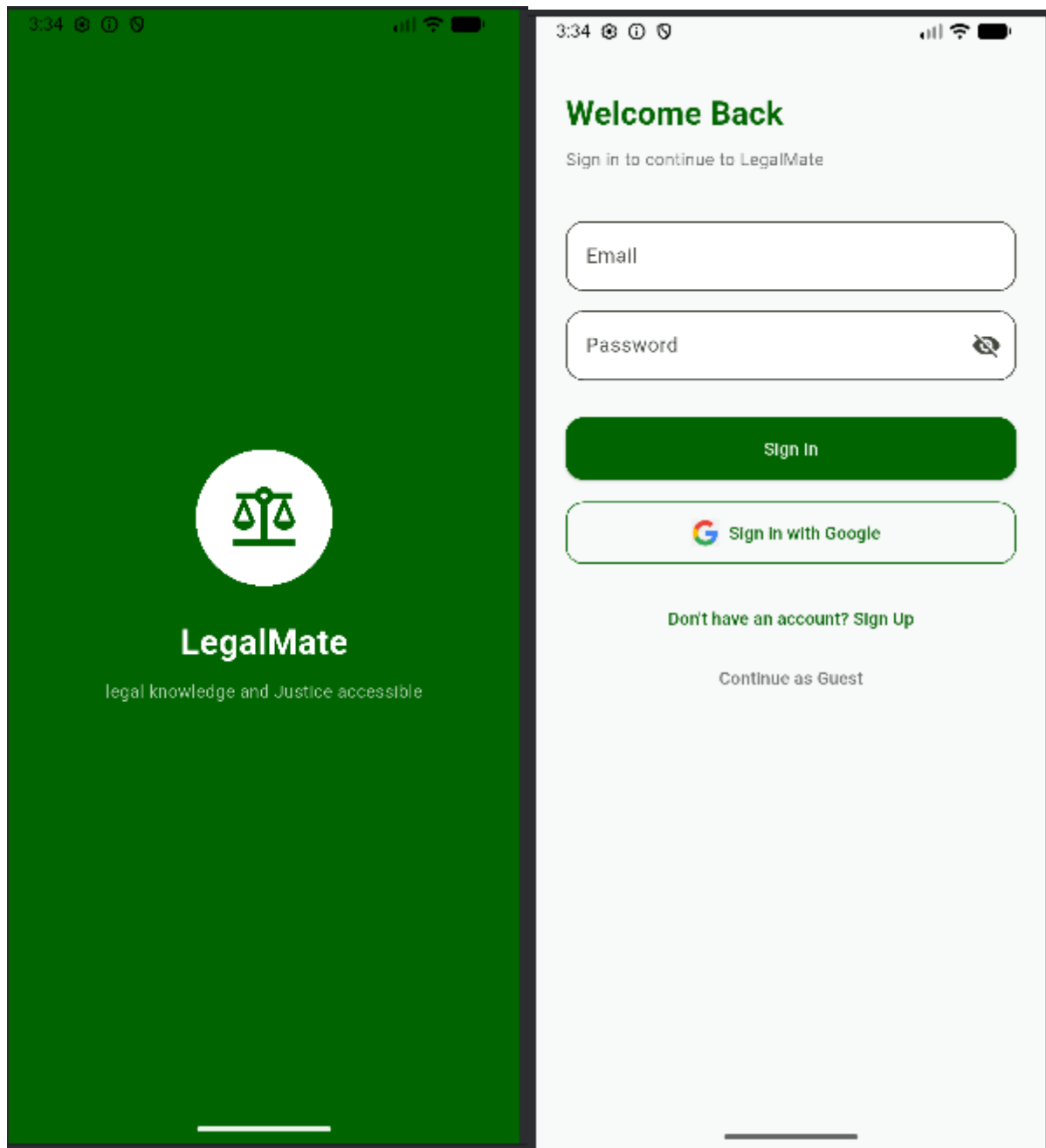


Figure 3.7: GUI 1

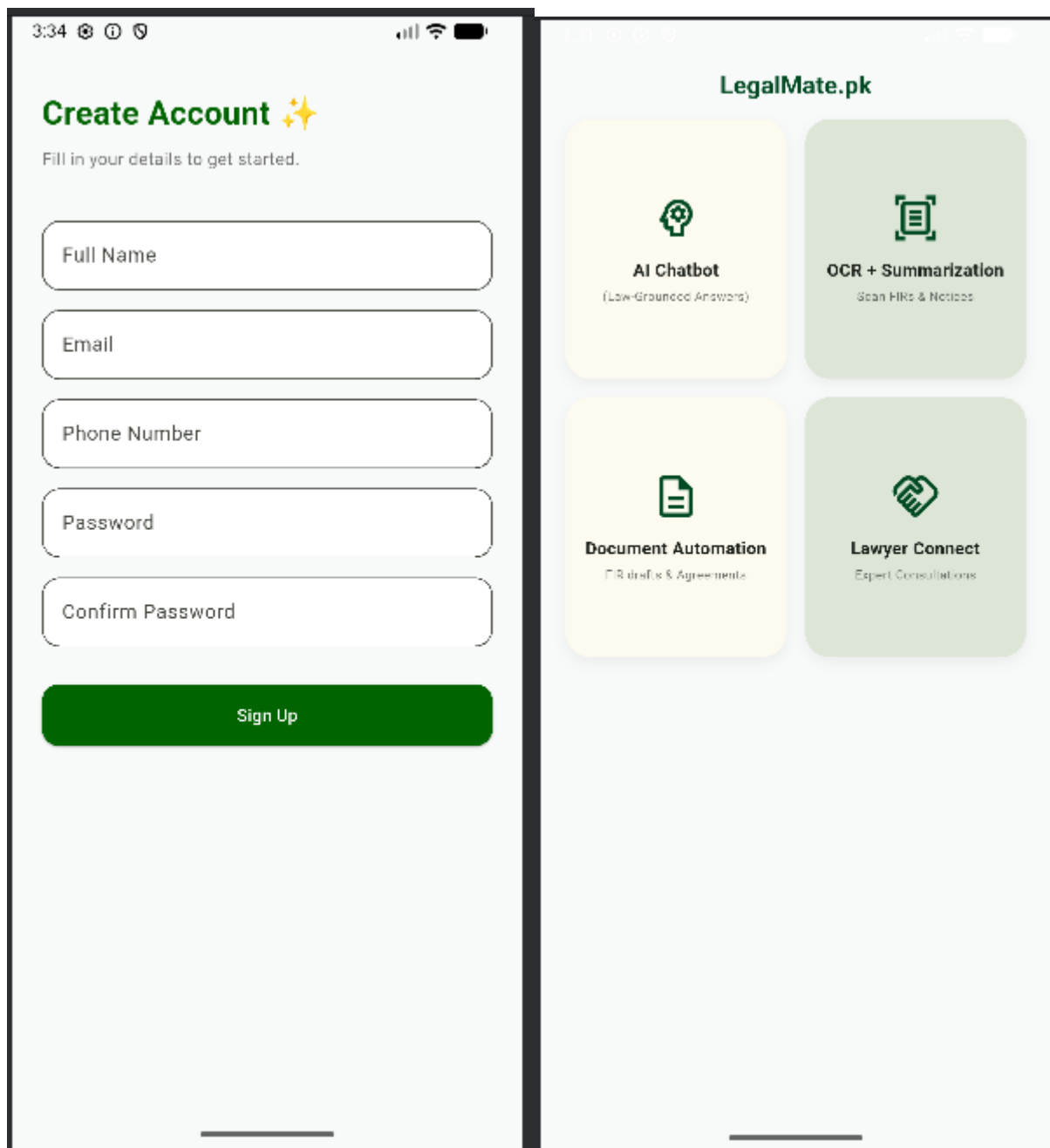


Figure 3.8: GUI 2

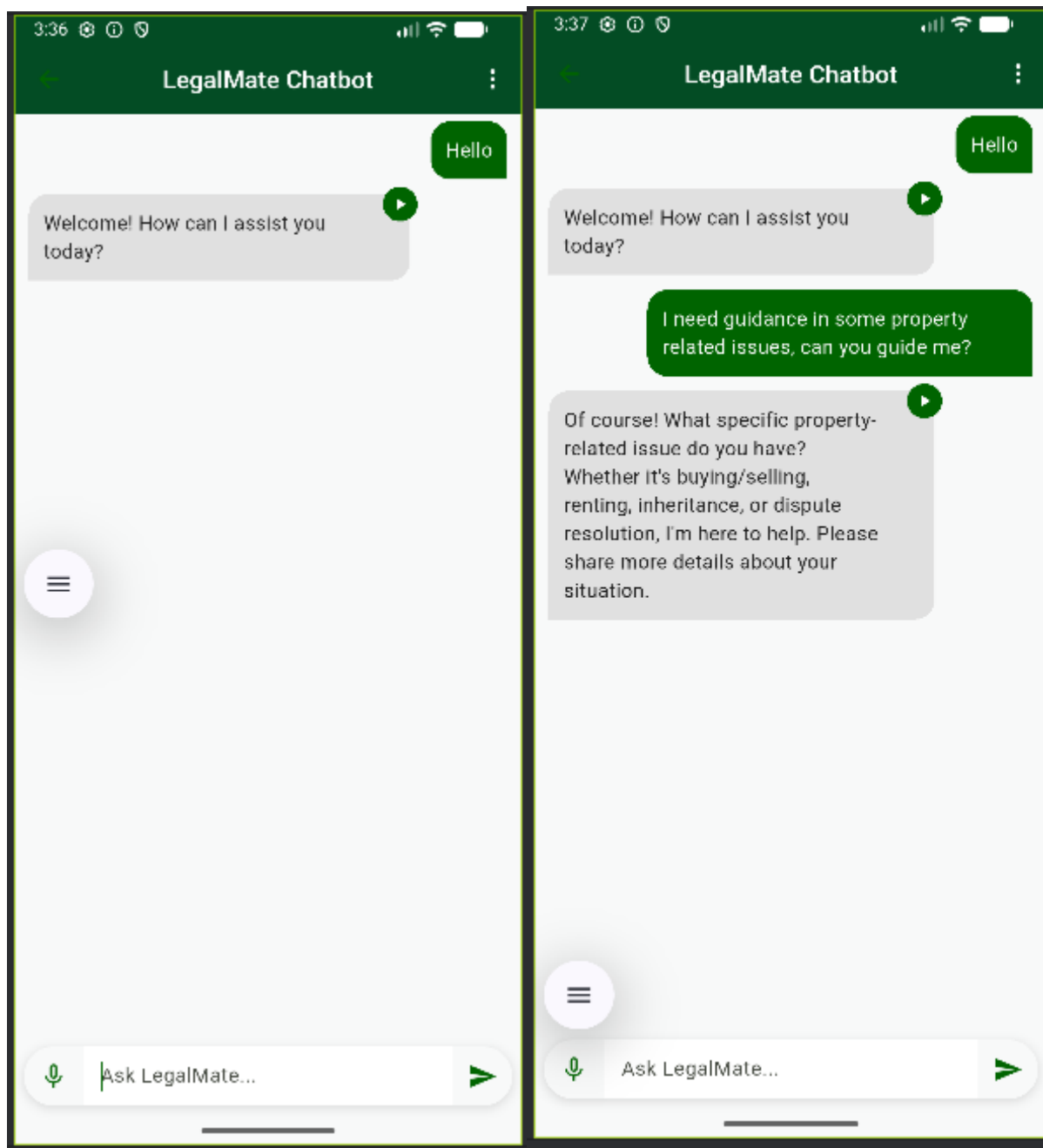


Figure 3.9: GUI 3

The form is a vertical stack of elements on a light gray background, flanked by dark gray vertical bars. It contains five rounded rectangular input fields for 'Full Name', 'Email', 'Phone Number', 'Password', and 'Confirm Password'. Below these is a role selector with two buttons: 'Citizen' (dark green) and 'Lawyer' (white with a green border). At the bottom is a large dark green 'Sign Up' button.

Full Name

Email

Phone Number

Password

Confirm Password

Citizen Lawyer

Sign Up

Figure 3.10: GUI 4

Chapter 4: Implementation and Test Cases

This chapter presents the implementation details of the Legal Mate system, focusing on the development of the chatbot module and its supporting components, like Document Generator, Lawyer portal and easy law explanation module. It explains the technologies used, the system setup, and the practical steps followed during implementation. In addition, this chapter discusses how the implemented modules were tested to ensure correct functionality. The goal of this chapter is to demonstrate how the proposed system design was translated into a working solution.

4.1 Implementation

Technology Stack Used

- **Frontend:** Flutter (Dart)
- **Backend:** Node.js
- **ML Service:** Python
- **Model Training & Experiments:** Google Colab, VS Code, Antigravity

4.1.1 User Interface Implementation (Current Status)

The user interface of the LegalMate.Pk application has been implemented using **Flutter**, allowing the system to be developed as a cross-platform mobile application. The current UI implementation focuses on providing basic navigation, user authentication, and interaction with the chatbot service.

The application is structured into multiple screens, each responsible for a specific functionality. These screens are implemented as separate Dart files to maintain modularity and ease of future enhancements.

The following screens have been implemented:

- **Splash Screen**

Displays the initial loading screen when the application starts.

- **Sign In Screen**

Allows existing users to log into the application.

- **Sign Up Screen**

Enables new users to create an account.

- **Home Screen**

Acts as the main navigation screen after successful login.

- **Chatbot Screen**

Provides the interface for users to interact with the LegalMate chatbot.

- **Legal Template Screen**

Displays available legal document templates for users.

- **OCR Screen**

Allows users to upload document images for OCR-based text extraction.

The application entry point is managed through **main.dart**, which initializes the app and handles routing between screens.

At the current stage, the UI is **functional but not finalized**. The primary goal of this phase was to establish the core screens and connect them with the backend and AI chatbot service. UI refinement, design improvements, and advanced user experience features will be completed in the next phase of the project.

4.1.2 Implementation of Chat-Bot Module

The Legal Mate chatbot was implemented using a combination of modern machine learning tools and web technologies. The overall system is divided into three main layers: user interface, backend services, and the machine learning service. The **user interface** is developed using **Flutter**, which allows the chatbot to be integrated into a mobile application. The **backend** is

Implementation and Test Cases

built using **Node.js**, which acts as an intermediary between the mobile application and the machine learning service. The **machine learning service** is implemented in **Python** and is responsible for processing user queries and generating responses using a fine-tuned language model.

Model training and experimentation were carried out using **Google Colab**, **VS Code**, and **Antigravity**, while the final training was executed on a local GPU-enabled system. This setup allowed efficient experimentation, debugging, and controlled fine-tuning of the model.

Hardware Environment for Fine-Tuning

The model fine-tuning process was performed on the following hardware and software environment:

- Operating System: Windows 10
- Python Version: 3.10.11
- PyTorch Version: 2.5.1 (CUDA enabled)
- GPU: NVIDIA GeForce RTX 3060
- GPU Memory: 12 GB
- CUDA Availability: Enabled

This setup provided sufficient computational power to fine-tune a 1.5B parameter model using parameter-efficient techniques.

Libraries and Frameworks Used

The following libraries were used during implementation and training:

- **transformers**, **datasets**, **accelerate** for model loading and training
- **bitsandbytes** for 4-bit quantization and QLoRA support
- **peft** for LoRA-based fine-tuning
- **tensorboard** and **matplotlib** for training visualization
- **rouge-score** and **nlk** for qualitative evaluation
- **wandb** (installed but disabled during final training)

Data Collection and Preparation

The chatbot was trained using legal data collected from multiple sources, including Kaggle, GitHub, Pakistani law websites, and Supreme Court judgment datasets. Since the raw data came in different formats such as PDFs and plain text, it could not be used directly for training.

The data underwent extensive cleaning and preprocessing to remove noise and inconsistencies. This included handling PDF pagination artifacts, removing superscripts and footnotes, fixing broken formatting caused by PDF conversion, correcting irregular spacing, resolving hyphenation issues from column scanning, restoring broken headings, and correcting punctuation errors. These steps ensured that the dataset was clean and suitable for training.

After cleaning, the data was converted into an **instruction-style format**, where each sample followed an instruction-input-output structure. This format was required to effectively fine-tune an instruction-following language model.

Dataset Balancing and Size Justification

The final training dataset used for fine-tuning the Legal Mate chatbot consisted of 5,000 instruction-response samples. The decision to limit the dataset to this size was intentional and well-justified, based on the model architecture, fine-tuning approach, domain focus, and hardware constraints.

- **Why a Large Dataset Was Not Required**

The base model used in this project, Qwen2.5-1.5B-Instruct, is a pre-trained large language model that has already been trained on billions of tokens of diverse text data. This pre-training enables the model to understand language structure, grammar, reasoning patterns, and general world knowledge. As a result, the purpose of fine-tuning in this project was not to teach the model language from scratch, but rather to adapt and specialize it toward the legal domain, specifically Pakistani property law.

In instruction fine-tuning, model performance depends more on the quality, clarity, and relevance of samples rather than sheer dataset size. A smaller, high-quality, domain-focused dataset is often more effective than a large, noisy, or generic dataset.

- **Role of QLoRA in Reducing Data Requirements**

Implementation and Test Cases

The fine-tuning process used QLoRA (Quantized Low-Rank Adaptation), a parameter-efficient fine-tuning technique. With QLoRA:

- The base model weights remain frozen
- Only a small number of LoRA adapter parameters are trained
- Memory usage is significantly reduced
- The model learns task-specific behavior, not general language patterns

In this setup, only around 18 million parameters were trainable out of more than 900 million total parameters. Because the number of trainable parameters is small, very large datasets are neither required nor desirable. Excessively large datasets can lead to:

- Overfitting to synthetic patterns
- Unstable training
- Diminishing performance gains

For LoRA-based fine-tuning, several studies and practical implementations have shown that a few thousand high-quality instruction samples are sufficient to achieve strong domain adaptation.

- **Instruction-Style Data Efficiency**

All samples were converted into a strict instruction-input-output format. This structure significantly increases data efficiency because:

- Each sample clearly defines the task intent
- The model learns how to follow instructions, not just generate text
- Ambiguity in supervision is reduced
- Training signals are stronger per sample

Compared to plain-text training, instruction-style data provides much higher learning value per example, making a dataset of 5,000 samples effectively comparable to a much larger unstructured dataset.

- **Domain Narrowing: Property Law Focus**

The Legal Mate chatbot is intentionally limited to property-related legal queries. The system is not designed to cover the entire legal domain. Because of this narrow domain scope, the dataset did not need to be extremely large. Instead of covering many unrelated legal topics, the dataset was carefully balanced to emphasize property law concepts, procedures, disputes, and terminology.

This focused approach ensures that:

- The model repeatedly sees relevant concepts
- Legal terminology is reinforced
- Responses remain consistent and accurate within scope

The final dataset distribution was:

- 60% Property Law
- 25% Supreme Court Judgments
- 10% Legal Question-Answer Data
- 5% General Question-Answer Data

This balance ensured specialization without losing general reasoning ability.

- **Effectiveness of Synthetic Data Augmentation**

Synthetic data generation was used only to supplement real legal data, not replace it. The seed-based generation approach ensured:

- Consistent instruction format
- Diverse legal scenarios
- No random or uncontrolled text generation

Because the synthetic data was generated using real legal examples as seeds, it preserved domain accuracy while expanding coverage. This made a 5,000-sample dataset both diverse and reliable.

- **Validation Split and Generalization**

Out of the 5,000 samples:

Implementation and Test Cases

- 4,500 were used for training
- 500 were used for validation

The validation loss consistently decreased during training, demonstrating that the model generalized well and did not overfit. This further confirms that the dataset size was sufficient and appropriate for the chosen model and fine-tuning strategy.

Table 4.1: Datasets Used

No.	Dataset Name	Source	Domain Focus	Original Format	Original Samples	Synthetic Data Used	Purpose in Training	Final Contribution (%)
1	Property Law Dataset	Kaggle, Pakistani Law Websites (PIDE, Mughal Barristers, Wirasat, NDMA)	Pakistani Property Law	PDF, Plain Text	411	Yes	Core domain knowledge for property laws, rights, procedures, and disputes	60%
2	Legal UQAD Dataset	GitHub	Legal Question Answering	Plain Text	990	Yes	Learning legal question/answer structure and legal reasoning	10%
3	Supreme Court Judgments Dataset	Kaggle	Judicial Decisions (Pakistan)	PDF, Plain Text	75	Yes	Exposure to real court language and judgment reasoning	25%
4	General UQAD Dataset	Hugging Face	General Question Answering	Plain Text	27	Yes	Maintaining general reasoning ability and	5%

							response fluency	
--	--	--	--	--	--	--	---------------------	--

Fine-Tuning Configuration

The chatbot was fine-tuned using the **Qwen2.5-1.5B-Instruct** model with **QLoRA**, which allows efficient training by updating only a small number of parameters. The training configuration included:

- Training samples: 5,000
- Training-validation split: 90% training, 10% validation
- Number of epochs: 2
- Learning rate: 2e-5
- Sequence length: 2048 tokens
- Gradient accumulation to simulate larger batch sizes
- Cosine learning rate scheduler with warmup
- LoRA configuration with rank 16, alpha 32, and dropout 0.1
- 4-bit quantization using NF4

Only around **18 million parameters** were trainable out of more than **900 million total parameters**, making the training process memory-efficient and stable.

Training Monitoring and Results

Training progress was monitored using loss values and validation performance. The training loss showed a steady decrease over time, indicating stable learning behaviour. Validation loss also improved across evaluation steps, confirming that the model was generalizing well and not overfitting.

Metrics such as BLEU and ROUGE were computed for qualitative evaluation to assess response quality, coherence, and relevance. Early stopping was guided by validation loss trends.

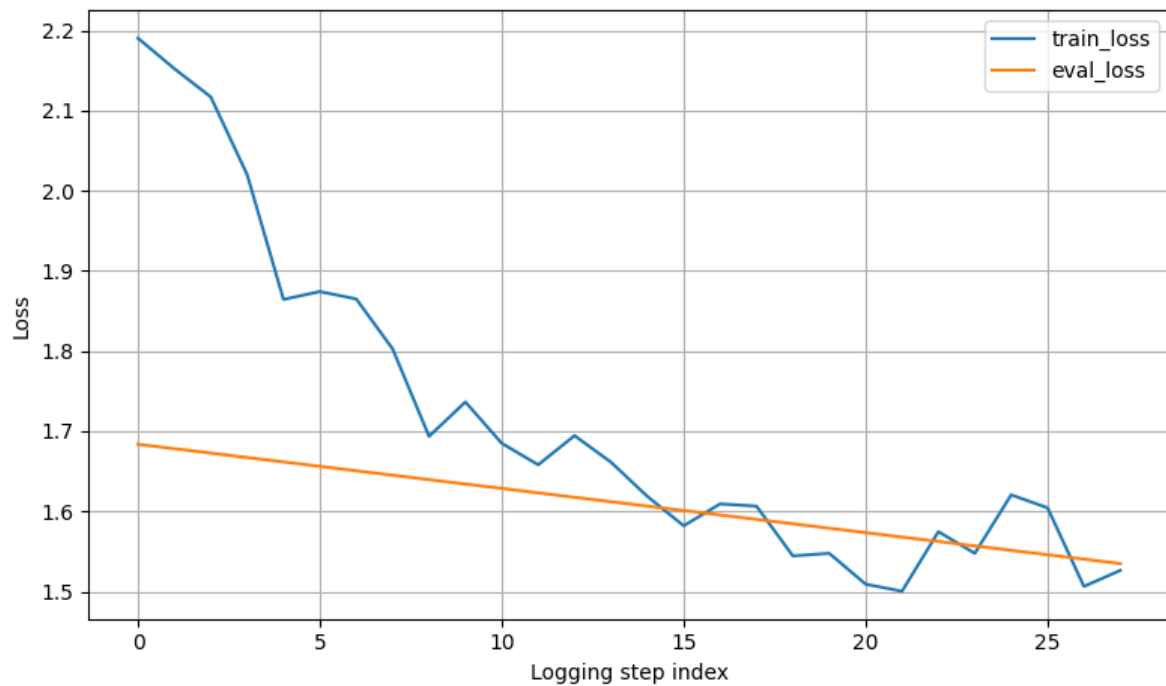


Figure 4.1: Training and Validation Loss

Conversation Management and Response Generation

The chatbot follows a **unified generative approach**, where intent recognition and response generation are handled by a single language model. Instead of relying on hardcoded rules or intent classifiers, the system uses **semantic understanding**. User queries are mapped into a high-dimensional vector space through the transformer's attention mechanism, allowing the model to understand legal intent contextually.

Responses are generated dynamically using the fine-tuned LoRA adapters. Controlled decoding strategies such as temperature and top-p sampling are used to balance factual accuracy and natural language flow. The chatbot operates in a **stateless manner**, meaning each query is processed independently. This design reduces memory overhead and improves user privacy by avoiding storage of conversation history on the server.

Underlying Model Concepts

The chatbot is based on a transformer architecture that uses self-attention to understand relationships between words in a sentence. During fine-tuning, transfer learning is applied by adapting the pre-trained model using LoRA, which injects task-specific knowledge while preserving the model's general language capabilities. The final output is generated based on contextual embeddings produced by the model's internal layers.

Chatbot Module Conclusion

The chatbot module of Legal Mate has been fully implemented and fine-tuned using a structured and efficient training pipeline. Through careful data preparation, synthetic data generation, and parameter-efficient fine-tuning, the chatbot is able to provide accurate and context-aware legal responses related to property law. This implementation forms a strong foundation for integrating additional modules such as OCR, voice input, and lawyer connect features in future stages of the project.

4.1.3 Implementation of RAG on Chat-Bot

The objective of implementing Retrieval-Augmented Generation (RAG) in the chatbot is to enable accurate, citation-grounded legal responses by combining a large external knowledge base with a fine-tuned language model. Traditional large language models (LLMs) were found insufficient for directly processing the project's legal dataset due to extreme input length, noisy OCR artifacts, and strict context-window limitations.

The source dataset consisted of 55 JSON objects, each containing:

- a filename key, and
- a text key holding very large OCR-extracted legal articles, including headings, subsections, references, and significant noise such as irregular spacing, repeated slashes (///), formatting symbols, and scanning artifacts.

Each text field exceeded the context window of standard LLMs, making direct ingestion infeasible.

Initial Challenges with Large-Context Processing

Several approaches were explored before arriving at the final solution:

1. Direct LLM Processing (Gemini / OpenAI UI)

Early attempts used general-purpose LLMs such as Gemini and OpenAI models through standard interfaces. These attempts failed because the OCR-extracted legal text was too large to fit within the available context window, causing upload and inference failures.

2. **Manual Chunking with Standard LLMs**

The text was manually divided into smaller chunks and processed sequentially using regular LLM prompts. However, this approach was extremely time-inefficient, with a single file requiring up to 7 hours to fully process. Additionally, the model lost global document context across chunks, resulting in inconsistent and fragmented outputs.

3. **API-Based Large Context Models**

API-based approaches using Gemini and OpenAI models were also attempted. Despite bypassing UI limitations, these models still failed due to context length constraints and were unable to reliably produce structured JSON outputs suitable for downstream RAG pipelines.

4. **OpenRouter Large-Context Models**

Large-context models accessed via OpenRouter were evaluated. While these models could ingest longer inputs, they struggled to consistently generate clean, schema-compliant structured JSON, which is a strict requirement for vector indexing and retrieval.

Adopted Processing Strategy (Context-Preserving Chunked Generation)

To overcome these limitations, a custom two-stage structured extraction strategy was implemented:

1. **Full-Context Reading Phase**

The selected large-context model was prompted to read the entire OCR text in a single pass, allowing it to internally retain global document context, section relationships, and legal hierarchies.

2. **Structured Output Generation in Controlled Chunks**

Instead of generating all outputs at once, the model was instructed to produce structured JSON outputs in batches of approximately 20 sections per chunk, while still referencing the full document context it had already consumed.

This approach successfully:

- Preserved semantic continuity across sections
- Avoided repeated context loss
- Produced clean, predictable structured JSON suitable for RAG

Each generated chunk contained:

- Section titles
- Cleaned legal text
- Source filename metadata

Vectorization and Storage in Pinecone

After structured data generation, the processed sections were uploaded to the vector database using Pinecone.

Key characteristics of this stage:

- Each legal section was stored as an independent vector entry
- Metadata included document name and section identifiers
- Pinecone's built-in embedding model was used, eliminating the need for an external embedding pipeline
- Embeddings were automatically generated during upsertion and indexed for semantic similarity search

At this stage, the system successfully completed **data ingestion and vector indexing**, forming the knowledge base required for RAG.

Planned Retrieval and Generation Flow (Not Yet Fully Integrated)

Although the vector database is populated, the RAG pipeline is not yet fully integrated into live chatbot inference. The intended runtime flow is as follows:

1. User Query Embedding

The user's query will be sent directly to Pinecone's embedding model to generate a query vector.

2. Semantic Retrieval

Pinecone will perform similarity search against the indexed legal vectors and return the most relevant legal sections.

3. RAG-Based Response Generation

Retrieved chunks will be passed as contextual references to the fine-tuned **Qwen 1.5B model**, enabling it to:

- Ground responses in retrieved legal text
- Generate accurate answers
- Include citations referencing the source sections

4. Frontend Delivery

The generated response, along with citations, will be forwarded to the frontend for user display.

4.2 Test case Design and description

This section will be added in FYP-II. Summarize the common attributes of test cases. This may include input constraints that must be true for every input in the set of associated test cases, any shared environmental needs, any shared special procedural requirements, and any shared case dependencies. The following scheme is recommended for describing test cases in detail.

4.2.1 Sample Test case No.1

<Software component Name>			
<Reference>			
Test Case ID:	<i>Reference Number</i>	Test Date:	<i>Date</i>
Test case Version:	<i>Version number</i>	Use Case Reference(s):	<i>Relation to use cases</i>
Revision History:	<i>Refer to previous test case identity (if any)</i>		
Objective	<i>Need and scope of the testing</i>		
Product/Ver/Module:	<i>Refer to overall system being built and the place of this test case in it.</i>		
Environment:	<i>Necessary and desired properties of the test environment. (hardware/software)</i>		

Assumptions:	<i>Assumptions that might affect the testing process.</i>	
Pre-Requisite:	<i>Necessary condition that needs to be fulfilled prior to the test case.</i>	
Step No.	Execution description	Procedure result
	<i>Events being tested.</i>	<i>Mention software response.</i>
Comments:		
☐ <i>Passed</i> ☐ <i>Failed</i> ☐ <i>Not Executed</i>		

4.2.2 Sample Test case No.2

.
.
.

4.3 Test Metrics

Summarize here the common ground of attributes of test case metrics.

4.3.1 Sample Test case Metric.No.1

Metric:	Purpose
Number of Test Cases:	Total number of test cases that you have developed for your system.
Number of Test Cases Passed:	The number of test cases that successfully passed
Number of Test Cases Failed:	The number of test cases that failed
Test Case Defect Density:	(No of test cases failed * 100) No of test cases executed
Test Case Effectiveness:	No of defects detected using test cases *100 Total number of defects detected
Traceability Matrix:	Traceability is the ability to determine that each feature has a source in requirements and each requirement has a corresponding implemented feature.

4.3.2 Sample Test case Metric.No.2

4.3.3 Sample Test case Metric.No.3

.
.
.
.

Chapter 5: Experimental Results and Analysis

This chapter will be added in FYP-II. Give proper analysis and discussion of experimental results (in plain English text) along with tables of results. **For each chapter provide a paragraph of introduction and in the end a paragraph of conclusions.**

Chapter 6: Conclusion and Future Directions

This chapter is mandatory. Give conclusions and summary of the work done. What were your findings and what were the results? Discuss in detail whether the scope of your project was entirely covered or not and whether the objectives of the project were met or not. What challenges did you face and what has been left out and why?

Sum up all the conclusions of all the chapters here to make a conclusion chapter. Do not repeat any text, just summarize it in different words.

Give recommendations for future work also. How your project can be further enhanced or improved? Future recommendations if someone wants to work on it. **For FYP-1 it is mandatory to list down a plan of the work to be done for FYP-2.**

References

- [1] A. Abdallah, B. Piryani, and A. Jatowt, "Exploring the state of the art in legal QA systems," *Journal of Big Data*, vol. 10, no. 1, p. 127, 2023. [Online]. Available: <https://doi.org/10.1186/s40537-023-00802-8>
- [2] "Legal Document Generation: Harnessing Large Language Models For Diverse Legal Workflows," *Indian Journal of Law and Legal Research*, vol. 6, no. 1, 2024. [Online]. Available: <https://www.ijllr.com/post/legal-document-generation-harnessing-large-language-models-for-diverse-legal-workflows>
- [3] J. Pattanashetti, A. Y., A. H., K. B., and P. K., "AI Legal Assistant," *International Research Journal of Modernization in Engineering Technology and Science*, vol. 7, no. 12, pp. 2704–2707, Dec. 2025. [Online]. Available: https://www.irjmets.com/upload_newfiles/irjmets71200098650/paper_file/irjmets71200098650.pdf
- [4] D. Panchal, A. Gole, V. Narute, and R. Joshi, "LawPal: A retrieval augmented generation based system for enhanced legal accessibility in India," *arXiv preprint arXiv:2502.16573*, Feb. 2025. [Online]. Available: <https://arxiv.org/abs/2502.16573>
- [5] T. Dong, M.-A. Nguyen, T. D. Hoang, N. T. Ngoc, D. X. Q. Minh, P. P. Hai, N. T.N. Anh, D. V. Tu, and B. Vu, "VLegal-Bench: Cognitively grounded benchmark for Vietnamese legal reasoning of large language models," *arXiv preprint arXiv:2512.14554v2*, Dec. 2025. [Online]. Available: <https://arxiv.org/abs/2512.14554v4>
- [6]

Appendix

Appendix A: Guidelines

This section should include all supporting information from the project that was not included in the body of the report. You should include surveys, complex statistical calculations, certain detailed tables and other such information in an appendix. The information presented in this section is important to support the work presented in the body of the report but would make it more difficult to read and understand if presented within the body of the report.

Cite the appendix items in the report narrative (write "see Appendix A") and organize appendices (e.g., Appendix A, Appendix B,

Any tables, figures, forms, or other materials that are not totally central to the analysis but that need to be included are placed in the Appendix.

Appendix B: Heading of Sample Appendix B

Following is a sample code with “code” style format.

```
Void SampleFunction(){  
    Print "Hello World.";  
}
```

Formatting Guidelines

This document also serves as style guide for final year project reports. In order to give a similar high-quality appearance to all final year software project reports this template uses a collection of predefined Microsoft Word formatting styles. **These styles should be used without modification or replacement.** Font in the document is “*Time New Roman*”. This template provides following styles:

- **Title** – the main title style
- **Title2** – the subtitle style
- **Body Text** – style for paragraphs
- **Caption** – the style for a figure or table caption
- **Table Description** – the style for description of table, it must be added after caption.
- **Figure Description** - the style for description of figure, it must be added after caption.
- **Code** – the style for program source code

```
int x = 10; // Writing important code
```
- **Table Header Row** – Style for the header row of table
- **Table Grid** – the style for the data rows in the tables
- **Reference** – The style for references
- **Bullets** – The style for the bullet lists
- **Numbered List**– Style for numbered lists

All Heading styles with different level numbers are listed below.

Chapter 1: Heading 1

1.1 Heading 2

1.1.1 Heading 3

1.1.1.1 Heading 4

1.1.1.1.1 Heading 5

1.1.1.1.1.1 Heading 6

1.1.1.1.1.1.1 Heading 7

1.1.1.1.1.1.1.1 Heading 8

1.1.1.1.1.1.1.1.1 Heading 9

Tables and Figures

Tables and figures should be centered horizontally. The caption button should be used to insert caption for both the figures and tables. All figures and tables must be numbered properly. Always refer to tables and figures according to their numbers. A table or figure can be cited as follows: ‘see Table1’ or ‘as shown in Table1’. The caption of table should be centered above the table and figure caption should be centered below the figure. Place the tables/figures close to their reference. Use “Table Header Row” and ‘Table Grid’ style for table’s header and data rows respectively. It is compulsory to provide brief description of table/figure after its caption. Styles for table and figure descriptions are “Table Description” and “Figure Description” respectively.

Press Ctrl+Shift+S to see list of styles mentioned above. Figure 1 shows the Apply Style window displaying the list of styles. Select any text then press Ctrl+Shift+S, the Apply Style window will show you the current style applied on that text and if required, you can change the style by selecting any other style from the “Style Name” dropdown.

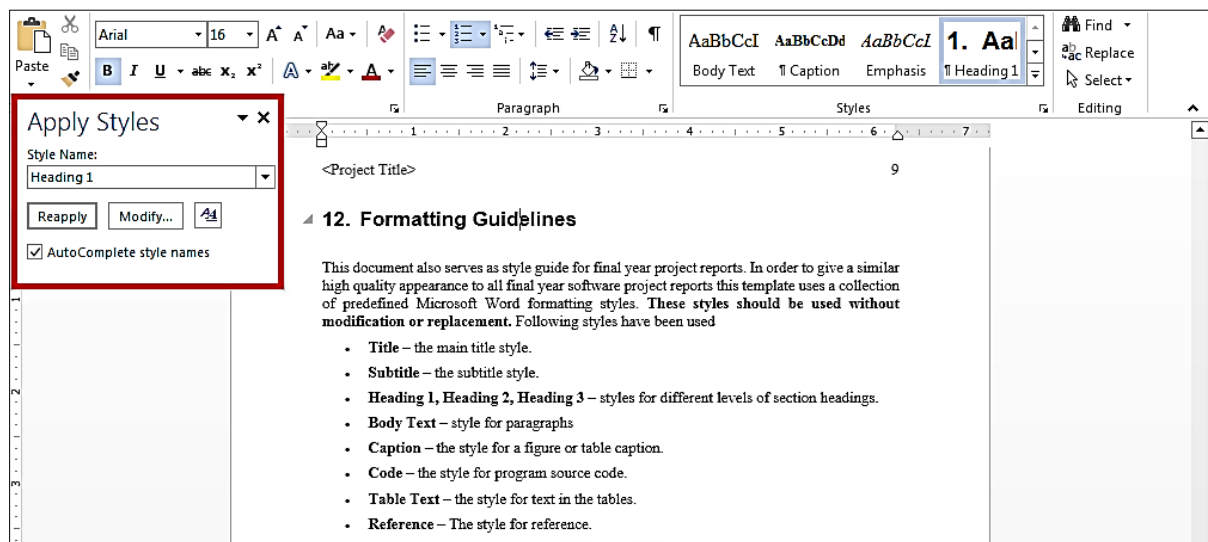


Figure 1.1: List of Styles

This is brief description of above figure.

Table 1.1: This is Sample table caption

This is brief description of following Table.

Header row	Header row	Header row	Header row
Row1 col1	Row1 col2	Row1 col3	Row1 col4
Row2 col1	Row2 col2	Row2 col3	Row2 col4

Table 1.2: This is Sample table caption

This is brief description of following Table.

Header row	Header row	Header row	Header row
Row1 col1	Row1 col2	Row1 col3	Row1 col4
Row2 col1	Row2 col2	Row2 col3	Row2 col4

Equations

Use equation editor to write equations in this report. Use last button of the custom tool bar to invoke equation editor. Similar to tables and figures, equations should also be aligned centered horizontally. Number all equations and insert them in parenthesis. Below is a sample equation and its reference number. An equation can be referenced like this: ‘it is clear from (1)’.

$$\sum_{\forall v \in V(G)} \deg(v) = 2|E(G)| \quad (1)$$

Header/Footer

Notice the headers in this document, before Introduction (i.e. the main content of this document) page numbers are in roman numerals. The page numbers of the actual content start with Arabic numerals i.e. 1, 2, 3 and so on. All of the **odd numbered pages** contain title of your project while the **even numbered pages** contain the section heading (i.e. chapter’s name) in the headers.

Other Formatting Guidelines

- Keep 2-4 GUIs in one page. Consume as much space as possible. Do not leave most of page blank unnecessarily.
- Do not break tables (or use cases) in multiple pages unless the table is too large to fit in one page.
- Re-arrange the content i.e., text, images, and tables properly to meet above two guidelines.

References

Always refer to the source of information by inserting the reference number in square brackets like this [5]. The reference numbers can either be added at the end of the sentence or within the sentence without changing the punctuation of sentence. A reference can also be cited as follows: ‘as Ruskey [2] mentioned’. List each source only once on your reference page.

- | |
|--|
| <ul style="list-style-type: none">[1] B. Klaus and P. Horn, <i>Robot Vision</i>. Cambridge, MA: MIT Press, 1986.[2] L. Stein, “Random patterns,” in <i>Computers and You</i>, J. S. Brake, Ed. New York: Wiley, 1994, pp. 55-70.[3] R. L. Myer, “Parametric oscillators and nonlinear materials,” in <i>Nonlinear Optics</i>, vol. 4, P. G. Harper and B. S. Wherret, Eds. San Francisco, CA: Academic, 1977, pp. 47-160.[4] M. Abramowitz and I. A. Stegun, Eds., <i>Handbook of Mathematical Functions</i> (Applied Mathematics Series 55). Washington, DC: NBS, 1964, pp. 32-33.[5] E. F. Moore, “Gedanken-experiments on sequential machines,” in <i>Automata Studies</i> (Ann. of Mathematical Studies, no. 1), C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton Univ. Press, 1965, pp. 129-153.[6] Westinghouse Electric Corporation (Staff of Technology and Science, Aerospace Div.), <i>Integrated Electronic Systems</i>. Englewood Cliffs, NJ: Prentice-Hall, 1970.[7] M. Gorkii, “Optimal design,” <i>Dokl. Akad. Nauk SSSR</i>, vol. 12, pp. 111-122, 1961 (Transl.: in L. Pontryagin, Ed., <i>The Mathematical Theory of Optimal Processes</i>. New York: Interscience, 1962, ch. 2, sec. 3, pp. 127-135).[8] G. O. Young, “Synthetic structure of industrial plastics,” in <i>Plastics</i>, vol. 3, <i>Polymers of Hexadromicon</i>, J. Peters, Ed., 2nd ed. New York: McGraw-Hill, 1964, pp. 15-64. |
|--|

Figure 1.2: IEEE Reference style

This figure represents the styling information for adding references in IEEE format

Following is a list of sample reference for various typed of sources in IEEE format.

[7]